# J.A.R.V.I.S

*Just A Rather Very Intelligent System*

Technical Build Documentation

**George Boateng**
Version 1.0 | March 2026
Built on Windows 10 | Python 3.14

# 1. Project Overview

J.A.R.V.I.S is a modular, voice-enabled personal AI assistant built in Python and powered by Anthropic's Claude API. Inspired by the fictional AI from the Iron Man franchise, this system is designed to automate everyday tasks, respond to natural language commands, and integrate with real-world services including Gmail, Google Calendar, smart home devices, and the local file system.

The assistant is built with a clean separation of concerns — each capability lives in its own module, making it easy to extend, debug, and maintain. The core brain is Claude, which classifies intent, routes commands, and generates natural responses.

## Key Capabilities

- Natural language conversation powered by Claude AI
- Voice input via Whisper speech-to-text (offline)
- Voice output via pyttsx3 text-to-speech (offline)
- Gmail integration — read and summarize unread emails
- Google Calendar — view and create events
- Smart home control via Home Assistant REST API
- File organization — auto-sort Downloads by file type
- Persistent memory across sessions using SQLite
- Secure credential storage using .env and Windows keyring

# 2. System Architecture

Jarvis follows a modular pipeline architecture. Every user interaction flows through five stages: input, classification, routing, execution, and output. This design makes each stage independently testable and replaceable.

## Pipeline Flow

| | |
|---|---|
| **Stage 1 — Input** | User speaks or types a command. In voice mode, audio is captured via sounddevice and transcribed by the Whisper model running locally on CPU. |
| **Stage 2 — Classification** | The raw text is sent to Claude (claude-sonnet-4-20250514) with a structured system prompt. Claude returns a JSON object containing the intent category, action, and any extracted parameters. |
| **Stage 3 — Routing** | The IntentRouter maps the classified intent to the correct module: email, calendar, smarthome, files, or general conversation. |

| | |
|---|---|
| **Stage 4 — Execution** | The matched module executes the action — calling an API, querying a database, or moving files. If no module matches, Claude handles it as a general conversation. |

| | |
|---|---|
| **Stage 5 — Output** | The response string is printed to the terminal and spoken aloud by the TTS engine. Both the user input and assistant response are saved to the SQLite memory database. |

### Folder Structure

| Component | Description |
|---|---|
| `core/` | Brain of the system: assistant orchestrator, intent router, memory manager |
| `modules/` | Automation modules: email, calendar, smart home, file organization |
| `voice/` | Speech-to-text (STT) and text-to-speech (TTS) engines |
| `utils/` | Shared utilities: secure config loader, centralized logger |
| `data/` | Runtime data: SQLite memory database, daily log files |
| `credentials/` | Google OAuth tokens (git-ignored, never committed) |
| `main.py` | Entry point — supports --text flag for text-only mode |
| `.env` | Secret API keys (git-ignored, stored locally only) |

## 3. Technology Stack

Every component was chosen for reliability, offline capability where possible, and compatibility with Windows 10.

| Layer | Tool | Purpose |
|---|---|---|
| **AI Brain** | Anthropic Claude API | Intent classification, routing decisions, and general conversation |
| **STT** | faster-whisper | Offline speech recognition using OpenAI's Whisper model (base size) |
| **TTS** | pyttsx3 | Offline Windows SAPI text-to-speech engine |
| **Audio Capture** | sounddevice + numpy | Low-latency microphone recording for voice input |
| **Gmail** | google-api-python-client | Read and manage emails via Gmail REST API |

| | | |
|---|---|---|
| **Calendar** | google-api-python-client | Create and list events via Google Calendar API |
| **Smart Home** | requests (REST) | Control devices via Home Assistant REST API |
| **File Watch** | watchdog | Monitor filesystem changes in real time |
| **Memory** | SQLite (built-in) | Persistent storage for conversations, tasks, preferences |
| **Secrets** | python-dotenv + keyring | Secure loading of API keys from .env and Windows Credential Manager |
| **Logging** | colorlog | Colored console output and rotating daily log files |
| **Scheduler** | schedule | Timed background tasks (reminders, briefings) |

# 4. Module Breakdown

## 4.1 core/intent_router.py

The intent router is the decision-making layer. It sends each user message to Claude with a structured system prompt that instructs the model to classify the input into one of five categories and return a JSON response.

**Intent categories:**

- email — reading, sending, or summarizing emails
- calendar — scheduling or checking calendar events
- files — organizing or searching the file system
- smarthome — controlling lights, switches, or scenes
- general — open-ended conversation or questions

Example Claude response for 'Turn off the bedroom lights':

```
{"intent": "smarthome", "action": "turn_off", "params": {"entity_id": "light.bedroom"}}
```

## 4.2 core/memory.py

Memory is stored in a local SQLite database at data/memory.db. Three tables are maintained:

- conversations — every user/assistant message with timestamp
- tasks — pending or completed tasks with due dates
- preferences — key-value store for user settings

The last 6–10 messages are retrieved and included in every Claude API call, giving the assistant short-term context awareness across the conversation.

## 4.3 modules/smarthome_module.py

Controls smart home devices via the Home Assistant REST API. Any device connected to Home Assistant — lights, switches, thermostats, plugs — can be controlled by voice command.

Supported actions: turn_on, turn_off, get_state, set_brightness. The entity_id parameter (e.g. light.living_room) is extracted by Claude from the user's natural language command.

## 4.4 modules/email_module.py

Connects to Gmail using OAuth 2.0. On first run, the user authorizes the app through a browser window. The access token is saved locally and reused on subsequent runs.

The read_unread action fetches the latest unread messages and returns sender and subject for each. Full email summarization using Claude can be added in a future iteration.

## 4.5 modules/calendar_module.py

Connects to Google Calendar using the same OAuth token as Gmail (both scopes are requested together). Supports listing upcoming events and creating new events with a title, start time, and duration.

## 4.6 modules/files_module.py

Organizes a target folder (default: Downloads) by moving files into subfolders based on their extension. Categories include documents, images, videos, audio, archives, and code.

Also supports a search action that recursively scans a folder tree for files matching a name pattern, returning up to 10 results.

## 4.7 voice/stt.py

Uses the faster-whisper library to run OpenAI's Whisper model locally on CPU. The 'base' model size is used by default — it offers a good balance of speed and accuracy. Larger models (small, medium, large) can be used for better accuracy at the cost of speed.

Audio is recorded for a fixed duration (default 5 seconds) using sounddevice, then passed to the Whisper transcription pipeline.

### 4.8 voice/tts.py

Uses pyttsx3, which wraps the built-in Windows SAPI voice engine. This runs fully offline with no API calls. Speech rate is set to 185 words per minute and volume to 90%. The first installed Windows voice is used by default.

## 5. Security & Credential Management

Jarvis handles several sensitive credentials: the Anthropic API key, Google OAuth tokens, and Home Assistant bearer tokens. A layered security approach is used:

- **API keys are stored in a .env file which is listed in .gitignore and never committed to version control**
- The .env file is loaded at runtime using python-dotenv
- Sensitive values can optionally be stored in Windows Credential Manager using the keyring library
- Google OAuth tokens are saved to credentials/google_token.json — also git-ignored
- No credentials are ever hardcoded in source files

**Best practice reminder:**

Never paste API keys into chat windows, terminals with visible history, or shared documents. Always store them in .env and add .env to .gitignore before pushing to any repository.

## 6. Installation & Setup

### Prerequisites

- Windows 10 or later
- Python 3.10 or later (3.14.3 used in development)
- Anthropic API key with active credits (console.anthropic.com)
- Optional: Google Cloud project with Gmail & Calendar APIs enabled
- Optional: Home Assistant instance on local network

### Step-by-Step Installation

**Step 1 — Navigate to the project folder:**

```
cd "C:\Users\George Boateng\Downloads\jarvis_project\jarvis"
```

**Step 2 — Create and activate virtual environment:**

```
python -m venv jarvis-env
jarvis-env\Scripts\activate
```

**Step 3 — Install dependencies:**

```
pip install anthropic faster-whisper pyttsx3 sounddevice numpy google-auth
pip install google-auth-oauthlib google-auth-httplib2 google-api-python-client
pip install requests python-dotenv keyring schedule watchdog colorlog
```

*Note: pyaudio is not required for text mode. It is only needed for voice mode and requires Microsoft C++ Build Tools to compile on Windows.*

**Step 4 — Add your Anthropic API key to .env:**

```
notepad .env
```

Replace the placeholder value with your real key from console.anthropic.com.

**Step 5 — Run Jarvis:**

```
python main.py --text
```

## Running Modes

| | |
|---|---|
| **--text mode** | Text-only input via keyboard. No microphone required. Recommended for initial setup and testing. TTS still speaks responses aloud. |

| | |
|---|---|
| **Voice mode** | Run python main.py without --text flag. Listens for 5 seconds per turn. Requires microphone and pyaudio installed. |

# 7. Example Commands

Once running with valid API credits, Jarvis responds to natural language. Some examples:

| Component | Description |
|---|---|
| `Read my emails` | Fetches unread Gmail messages and lists sender + subject |
| `What's on my calendar today?` | Lists upcoming Google Calendar events |
| `Schedule a meeting tomorrow at 3pm` | Creates a new calendar event |
| `Turn off the living room lights` | Sends turn_off command to Home Assistant entity |
| `Set bedroom lights to` | Sets brightness to ~128/255 via Home Assistant |

| | |
|---|---|
| `50%` | |
| `Organize my downloads folder` | Sorts files by type into subfolders |
| `Find files named report` | Searches Downloads recursively for matching files |
| `Tell me a joke` | Handled by Claude as general conversation |
| `Goodbye Jarvis` | Graceful shutdown with farewell message |

# 8. Future Roadmap

The following features are planned for future versions:

- Wake word detection (e.g. 'Hey Jarvis') using pvporcupine or openWakeWord
- Daily briefing mode — scheduled morning summary of emails, calendar, and weather
- Email summarization using Claude to condense long email threads
- Reminder system with desktop notifications
- Web search integration for real-time information lookup
- GUI dashboard built with tkinter or a web frontend
- Mobile access via a simple Flask or FastAPI server
- Multi-user support with per-user memory profiles
- Plugin system for third-party module contributions

# 9. Known Limitations

- faster-whisper and the av/PyAV dependency are not yet fully compatible with Python 3.14. Voice mode requires Python 3.11 or earlier for stable operation.
- pyaudio requires Microsoft C++ Build Tools to compile from source on Windows. Use the --text flag to bypass this requirement.
- pipwin (a helper for installing Windows audio packages) is incompatible with Python 3.14 due to js2py bytecode changes.
- The intent router makes one Claude API call per message for classification, plus a second call for general conversation responses. This doubles API usage for non-module queries.
- Google OAuth tokens expire and may need to be refreshed by re-running the setup_oauth() method.

# 10. Glossary

| | |
|---|---|
| **Claude API** | Anthropic's large language model API used for intent classification and natural language responses |
| **Intent Router** | The component that sends user input to Claude and receives a structured JSON classification back |
| **STT** | Speech-to-Text — converting spoken audio into written text (handled by Whisper) |
| **TTS** | Text-to-Speech — converting written text into spoken audio (handled by pyttsx3) |
| **OAuth 2.0** | Authorization protocol used to grant Jarvis access to Gmail and Google Calendar without storing passwords |
| **Home Assistant** | Open-source home automation platform that exposes a REST API for controlling connected devices |
| **Entity ID** | Home Assistant identifier for a device, e.g. light.living_room or switch.bedroom_fan |
| **SQLite** | Lightweight file-based relational database used to store conversation history and preferences |
| **Virtual Env** | Isolated Python environment (jarvis-env/) that keeps project dependencies separate from system Python |
| **.env file** | Plain text file containing secret environment variables like API keys — never committed to Git |

# End of Documentation

*J.A.R.V.I.S v1.0 — George Boateng — March 2026*